Guía de repaso para el Examen 2

Listas enlazadas, Ley de Deméter, matrices, herencia y polimorfismo

EIF201 Programación I · II Ciclo 2026 · Tutoría del lunes 2 de noviembre
Andrey Jesús Solano Rojas, tutor

1. Qué entra en el examen

El Examen 2 (domingo 8 de noviembre, 1:00 p. m.) es acumulativo y se concentra en los temas vistos después del Examen 1, según el cronograma de la carta al estudiante. Esta guía resume lo esencial de cada uno y termina con cómo se combinan en un problema típico de examen.

Tema	Semana	Qué hay que dominar
Herencia	9	Constructor del padre, `protected`, sobrescritura, orden de constructores y destructores.
Listas enlazadas	10–11	Insertar, recorrer, buscar, eliminar y sustituir, con todos los casos especiales.
Ley de Deméter y principios	13	Reconocer y corregir cadenas de llamadas; cohesión, acoplamiento, responsabilidad única.
Matrices	13–14	Recorridos; matriz dinámica de punteros a objetos: crear, validar, liberar.
Polimorfismo	15	`virtual`, clases abstractas, destructor virtual, colecciones polimórficas, `dynamic_cast`.
Memoria dinámica	Todas	Cada `new` con su `delete`; saber quién es responsable de cada objeto.

2. Listas simplemente enlazadas

Invariantes

- `primero` apunta al primer nodo, o es `nullptr` si la lista está vacía.
- El último nodo tiene `sig == nullptr`.
- Cada nodo creado con `new` se libera exactamente una vez.

Toda operación debe dejar estos invariantes intactos en **todos** los casos: lista vacía, un solo nodo, elemento al inicio, en medio, al final e inexistente.

Condiciones de recorrido

Condición	`actual` termina en	Requiere
`actual != nullptr`	después del último	nada
`actual->getSig() != nullptr`	el último	lista no vacía
`actual->getSig()->getSig() != nullptr`	el penúltimo	al menos dos nodos

Eliminar: localizar, reenlazar, liberar

```
bool Lista::eliminar(int valor) {
    Nodo* anterior = nullptr;
    Nodo* actual = primero;
    while (actual != nullptr && actual->getDato() != valor) {
        anterior = actual;
        actual = actual->getSig();
    }
    if (actual == nullptr)              // vacía o no existe
        return false;
    if (anterior == nullptr)            // era el primero
        primero = actual->getSig();
    else                                // en medio o al final
        anterior->setSig(actual->getSig());
    delete actual;
    return true;
}
```

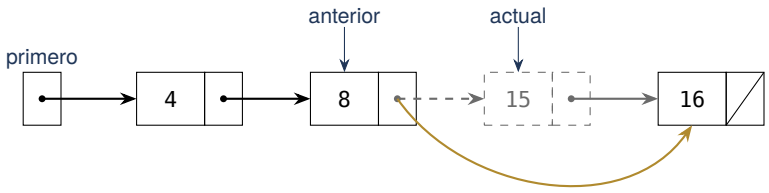


Figura 1. `eliminar(15)`: el anterior pasa a apuntar al siguiente del eliminado.

Listas de objetos

Si cada nodo apunta a un objeto (`Producto* info`), hay que decidir quién lo libera. Si la lista es responsable, al eliminar se libera primero el objeto y luego el nodo. Si se *extrae* un objeto para pasarlo a otra estructura, se libera solo el nodo y se retorna el puntero al objeto.

3. Ley de Deméter y principios de diseño

La Ley de Deméter (principio de menor conocimiento) dice que un método solo debe conversar con sus "vecinos inmediatos": sus atributos, sus parámetros y los objetos que crea. Una cadena de llamadas como esta la viola:

```
// Viola la Ley de Deméter: el hospital conoce la estructura interna del pabellón
cout << hospital->getPabellon()->getCama(2, 3)->getPaciente()->getNombre();

// Corrección: cada clase ofrece la operación y delega
cout << hospital->nombrePacienteEn(2, 3);
```

Principio	En una frase
Alta cohesión	Todo lo que hace una clase está relacionado con un mismo propósito.
Bajo acoplamiento	Las clases dependen lo menos posible de los detalles internos de otras.
Responsabilidad única	Una clase tiene una sola razón para cambiar.
Abierto-cerrado	Se puede extender (agregar derivadas) sin modificar el código que ya funciona.
Ley de Deméter	No encadenar llamadas a través de objetos ajenos; delegar.

4. Matrices

Recorridos

Recorrido	Patrón
Por filas	`for f { for c { m[f][c] } }`
Por columnas	`for c { for f { m[f][c] } }`
Una fila / una columna	`f` fijo y se mueve `c` / `c` fijo y se mueve `f`
Diagonales (cuadrada)	`m[i][i]` y `m[i][n-1-i]`

Matriz dinámica de punteros a objetos

Tres niveles: `Tipo***` (arreglo de filas), `Tipo**` (una fila) y `Tipo*` (una celda, que apunta a un objeto o es `nullptr`). Se crea de arriba hacia abajo y se libera en orden inverso:

```
SalaCine::SalaCine(int filas, int columnas) : filas(filas), columnas(columnas) {
    asientos = new Reserva**[filas];              // 1. arreglo de filas
    for (int f = 0; f < filas; f++) {
        asientos[f] = new Reserva*[columnas];     // 2. cada fila
        for (int c = 0; c < columnas; c++)
            asientos[f][c] = nullptr;             // 3. asiento libre
    }
}
```

```
SalaCine::~SalaCine() {
    for (int f = 0; f < filas; f++) {
        for (int c = 0; c < columnas; c++)
            delete asientos[f][c];                // 1. objetos
        delete[] asientos[f];                     // 2. cada fila
    }
    delete[] asientos;                            // 3. arreglo de filas
}
```

- Las celdas **siempre** se inicializan en `nullptr`.
- Toda operación que recibe `(f, c)` valida la posición antes de acceder.
- Después de liberar una celda se deja en `nullptr`.
- Mover un objeto de celda es copiar el puntero y dejar el origen en `nullptr`, sin `new` ni `delete`.

5. Herencia

- **Relación "es un".** Si la frase natural es "tiene un", no es herencia.
- **Constructor del padre** en la lista de inicialización: `Hija(...) : Padre(...), x(...) {}`.
- **Orden:** constructores de la base a la derivada; destructores al revés.
- **`protected`:** accesible para las derivadas, no desde fuera.
- **Sobrescritura:** por *sustitución* (reemplaza al padre) o por *extensión* (`Padre::metodo()` más algo). Olvidar `Padre::` al extender produce recursión infinita.

6. Polimorfismo

Concepto	Lo esencial
Enlace estático	Método *no* virtual llamado con puntero a la base: se ejecuta según el tipo del *puntero*.
Enlace dinámico	Método `virtual`: se ejecuta según el tipo real del *objeto*.
Rebanado	Copiar un objeto derivado a uno base por valor pierde lo específico. El polimorfismo requiere punteros o referencias.
Virtual puro	`virtual double f() const = 0;` Obliga a las derivadas a implementarlo.
Clase abstracta	Tiene al menos un virtual puro. No se pueden crear objetos, sí punteros.
Clase interface	Solo métodos virtuales puros y destructor virtual, sin atributos.
Destructor virtual	Obligatorio en toda base polimórfica; si no, `delete` por puntero a la base no ejecuta el destructor de la derivada.
`override`	Pide al compilador verificar la sobrescritura. Detecta firmas distintas (por ejemplo, sin `const`).
Sobrecarga	Mismo nombre, distintos parámetros, en la misma clase. Se decide al compilar.

Colecciones polimórficas y `dynamic_cast`

Una colección guarda punteros a la base (`Base**` en un arreglo, `Base***` en una matriz, `Base* info` en una lista). Para lo que todos tienen, se usa un método virtual; para lo que solo tiene una derivada, `dynamic_cast`:

```
double Planilla::totalBonos() const {
    double total = 0;
    for (int i = 0; i < cantidad; i++) {
        Gerente* g = dynamic_cast<Gerente*>(empleados[i]);
        if (g != nullptr)               // solo los gerentes tienen bono
            total += g->getBono();
    }
    return total;
}
```

- `dynamic_cast` retorna `nullptr` si el objeto no es de ese tipo (o si el puntero ya era `nullptr`). **Siempre** se verifica antes de usar el resultado.
- Responde "¿se puede tratar como...?": convertir un `Gerente` a `EmpleadoFijo*` funciona, porque un gerente *es un* empleado fijo.
- `static_cast` no verifica nada: con el tipo equivocado produce un puntero inválido.
- Con referencias, si falla lanza `std::bad_cast`.

7. Cómo se combinan en un problema de examen

Un problema típico mezcla varias estructuras y obliga a decidir quién es responsable de cada objeto. En el simulacro de esta tutoría, un pabellón de hospital tiene una **lista enlazada de espera** y una **matriz de camas**, y ambas guardan punteros a una **jerarquía abstracta** de pacientes:

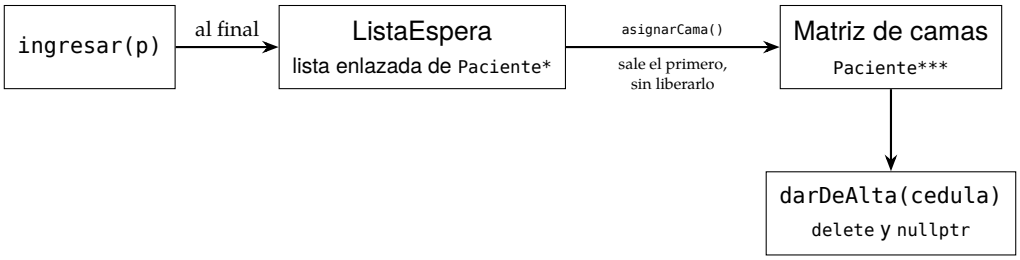


Figura 2. El paciente pasa de la lista a la matriz: la responsabilidad se transfiere, no se copia.

- Al pasar de la lista a la cama se libera el *nodo*, pero no el paciente: ahora la matriz es responsable.
- Antes de sacar a alguien de la lista hay que verificar que exista una cama libre; si no, se perdería.
- El costo total es polimórfico (`calcularCosto()` virtual) y para contar las urgencias se usa `dynamic_cast`.
- El destructor del pabellón libera la matriz; el de la lista libera a quienes siguen esperando.

8. Errores frecuentes

Error	Corrección
No actualizar `primero` al eliminar el primer nodo	Distinguir `anterior == nullptr`.
Liberar un nodo antes de reenlazar	Localizar, reenlazar, liberar.
Correr los punteros de un arreglo antes de liberar el elemento	Liberar primero, correr después.
No inicializar la matriz en `nullptr`	Tercer paso del constructor.
Liberar el arreglo de filas antes que las filas	Objetos, filas, arreglo.
`delete` en lugar de `delete[]`	`delete[]` para todo `new[]`.
No validar `(f, c)`	Validar al inicio de cada método.
No llamar al constructor del padre	`: Padre(args)` en la lista de inicialización.
Firma distinta en la derivada (sin `const`)	Usar `override`.
Destructor de la base sin `virtual`	`virtual ~Base() {}`.
Usar `dynamic_cast` sin verificar	Comparar con `nullptr`.
Cadenas de llamadas entre objetos ajenos	Agregar un método que delegue (Deméter).

9. Lista de verificación para el examen

- ☐ Leer todo el enunciado antes de empezar y repartir el tiempo según el valor de cada parte.
- ☐ Dibujar el UML: clases, herencia, composición o agregación, multiplicidad.
- ☐ Dibujar la memoria (nodos, matriz, objetos) antes de programar cada método.
- ☐ Para cada método que recibe una posición o un valor: ¿qué pasa si no existe, si está vacío, si está fuera de rango?
- ☐ Para cada `new`: ¿quién hace el `delete` y cuándo?
- ☐ Toda clase base polimórfica tiene destructor virtual.
- ☐ Todo `dynamic_cast` se verifica antes de usarse.

Material de referencia de la tutoría

Guía teórica *Listas simplemente enlazadas* (semana 11).

Guía teórica *Herencia y polimorfismo* (semanas 9 y 15).

Guía teórica *Matrices de objetos, herencia y downcasting* (semana 14).